PHYS551: Advanced Monte Carlo Methods

Homework 3

Student:
Ömer Faruk Avci

Instructor:
Prof. Dr. Taylan Akdoğan

Department of Electrical and Electronics Engineering

March 18, 2026

Disclosures and References

AI Usage Disclosure: In accordance with the course policy, I would like to declare the extent of AI assistance used in this assignment:

- **Analytical Derivations:** While the core logical steps were performed by me, AI tools were used to verify certain mathematical derivations and to assist in the formal LaTeX formatting of equations.
- **Code Implementation:** The code was developed by me. AI tools were consulted to optimize the grid search logic, verify the variance calculation, and refine code comments.
- **Report Writing:** The explanations and discussions were written by me. AI tools were used to check for grammatical errors and to give feedback on the overall structure.

1) Analytical Definition of the Proposal Distribution

Let $u = v - 45$. The proposal distribution $q(v)$ is a **Shifted Gamma distribution**:

$$q(v) = \frac{\beta^\alpha}{\Gamma(\alpha)} (v - 45)^{\alpha-1} e^{-\beta(v-45)}, \qquad v \geq 45 \tag{1}$$

with parameters $\alpha = k = 4.2894$ and $\beta = 1/\theta = 1/0.9519$, i.e. $\theta = 0.9519$.

Analytical Verification

Substituting $u = v - 45$ (so $du = dv$):

$$\int_{45}^{\infty} q(v)\, dv = \frac{\beta^\alpha}{\Gamma(\alpha)} \int_0^\infty u^{\alpha-1} e^{-\beta u}\, du \tag{2}$$

The integral $\displaystyle\int_0^\infty u^{\alpha-1} e^{-\beta u}\, du$ is the standard Gamma integral. To evaluate it analytically, apply the substitution $t = \beta u$, so $u = t/\beta$ and $du = dt/\beta$:

$$\int_0^\infty u^{\alpha-1} e^{-\beta u}\, du = \int_0^\infty \left(\frac{t}{\beta}\right)^{\alpha-1} e^{-t}\, \frac{dt}{\beta} = \frac{1}{\beta^\alpha} \int_0^\infty t^{\alpha-1} e^{-t}\, dt = \frac{\Gamma(\alpha)}{\beta^\alpha} \tag{3}$$

where $\Gamma(\alpha) = \int_0^\infty t^{\alpha-1} e^{-t}\, dt$ is the Gamma function. For positive integer α, this reduces to $\Gamma(\alpha) = (\alpha - 1)!$, provable by repeated integration by parts. Substituting back:

$$\int_{45}^{\infty} q(v)\, dv = \frac{\beta^\alpha}{\Gamma(\alpha)} \cdot \frac{\Gamma(\alpha)}{\beta^\alpha} = 1 \tag{4}$$

The normalization constant $\beta^\alpha/\Gamma(\alpha)$ is therefore exact. The full set of parameters is:

$$\boxed{\alpha = k = 4.2894, \qquad \beta = \frac{1}{\theta} = \frac{1}{0.9519} \approx 1.0526, \qquad \text{shift} = 45} \tag{5}$$

2) Justification

Analyzing the shape of $f(v) = C(v) \cdot p(v)$

The integrand is:

$$f(v) = 10^5 (v-45)^3 \cdot \frac{v}{50} e^{-v^2/100}, \qquad v \geq 45$$

By numerically plotting $f(v)$ over the range $v \in [45, 80]$, the following geometric properties were identified:

- $f(45) = 0$ because $(v-45)^3 = 0$ at the boundary.
- $f(v)$ rises from zero, reaches a peak near $v \approx 48.19$ (i.e., $u^* = v - 45 \approx 3.19$), and then decays with a right-skewed tail due to the Gaussian factor $e^{-v^2/100}$.
- The half-maximum width of this hump is approximately $\Delta v \approx 4.17$ m/s.

Stage 1: Parameter Derivation

Parameters were first estimated analytically by matching the mode and variance of the Gamma to the measured properties of $f(v)$.

Mode matching: The mode of a Gamma distribution (for $u = v - 45$) is:

$$(k-1) \cdot \theta = u^* = 3.19$$

Variance matching: Using the Gaussian FWHM approximation $\Delta v \approx 2\sqrt{2 \ln 2}\, \sigma \approx 2.355\, \sigma$:

$$\sigma \approx \frac{4.17}{2.355} \approx 1.77 \implies \sigma^2 \approx 3.14$$

and since $\mathrm{Var}[\mathrm{Gamma}(k, \theta)] = k\theta^2$:

$$k \cdot \theta^2 = 3.14$$

Solving this system simultaneously by substituting $\theta = 3.19/(k-1)$ into the variance equation yields the quadratic:

$$3.14k^2 - 16.46k + 3.14 = 0$$

with solution $k \approx 4.982$, $\theta \approx 0.799$.

Stage 2: Parameter Refinement and Optimization

The initial geometric parameters from Stage 1 provided a baseline, but the resulting proposal was too narrow to effectively cover the right-skewed tail of the integrand. To minimize the contest score, I implemented a two-stage optimization process.

Coarse Grid Search (Stage 2): First, a coarse grid search was performed over $\theta \in [0.70, 1.20]$ with 20 steps. At this stage, the mode constraint $(k-1)\theta = u^*$ was strictly maintained to isolate the effect of the distribution's width. To suppress Monte Carlo noise during the search, the score for each configuration was averaged over three independent runs. This identified a clear local minimum near $\theta \approx 0.940$ and $k \approx 4.385$, yielding a score of $\approx 1.96 \times 10^{-9}$.

Fine Cross Search (Stage 3): Once the high-performance region was identified, the strict mode constraint was relaxed. Because the integrand is asymmetrical, small deviations from the theoretical mode can actually reduce variance by providing better "weight" to the heavier tails. A fine-resolution cross search was conducted independently over both k and θ:

- θ range: $[0.890, 0.990]$ with a step size of 0.005
- k range: $[4.235, 4.535]$ with a step size of 0.005

- All configurations were verified using 3-run averaging for stability.

The fine search identified an optimal parameter set at $k = 4.2910$ and $\theta = 0.9520$, improving the score by another factor of $1.77\times$. For the final submission, I selected the nearby rounded values of $k = 4.2950$ and $\theta = 0.9500$, which were verified to provide a consistent score of approximately 1.1×10^{-9} across multiple runs.

Optimization Stage	k	θ	**Mode** $(k-1)\theta$	**Contest Score**
Stage 1 (Geometric)	4.982	0.7990	3.2279	$\sim 2.5 \times 10^{-8}$
Stage 2 (Coarse Grid)	4.3850	0.9400	3.1819	$\sim 1.96 \times 10^{-9}$
Stage 3 (Final Optimized)	4.2950	0.9500	3.1303	$\sim \mathbf{1.14 \times 10^{-9}}$

Table 1: Evolution of parameters and variance reduction through successive optimization stages.

3) Sampling Strategy

To generate the $N = 100,000$ samples required for the simulation, I used a translation-based sampling approach to transform standard Gamma variables into the target domain of $v \geq 45$. The procedure is as follows:

- **Random Variable Generation:** I generated N independent random variables, u_i, from a standard Gamma distribution using the optimized shape parameter $\alpha = k = 4.2894$ and scale parameter $\theta = 0.9519$.
- **Domain Shifting:** Since the structural yield limit and the resulting financial damage only occur for wind speeds $v \geq 45$ m/s, each generated Gamma variable was shifted to the right by the location parameter $v_{\text{yield}} = 45$.
- **Final Transformation:** The final samples for the wind speed v were calculated as $v_i = u_i + 45$.
- **Implementation:** This was implemented in Python using the `numpy.random.gamma(shape, scale, size)` function. This method draws samples analytically, avoiding the need for numerical inversion of the CDF as prohibited by the contest rules.

By using this strategy, the samples are automatically concentrated around the calculated "hump" of the integrand ($v \approx 48.19$ m/s) rather than at the boundary, ensuring high sampling efficiency for these rare-event scenarios.

4) Implementation

Python Source Code for Problem 1

```python
"""

# PHYS 551: Advanced Monte Carlo Methods
# Homework 3 - Source Code
# Student: Omer Faruk Avci
# Instructor: Prof. Dr. Taylan Akdogan

"""
import numpy as np
from scipy import stats
from scipy.stats import skew, kurtosis


"""Problem 1: Offshore Wind & Hurricane Risk"""
```

```python

N = 100_000
shift = 45
u_star = 3.1817 # u*: used contraint for Phase 1
theta = 4.982 # Found analitically at the report
k = 0.799 # Found analitically at the report

def simulate_IS(k, theta):
    V = shift + np.random.gamma(k, theta, N)
    p_v = (V / 50) * np.exp(-(V**2) / 100)
    C_v = 10**5 * (V - 45)**3
    q_v = stats.gamma.pdf(V - shift, a=k, scale=theta)
    weighted_evals = C_v * (p_v / q_v)

    eal_est = np.mean(weighted_evals)
    score = np.var(weighted_evals, ddof = 1)

    return eal_est, score

# ============================================================
# PHASE 1: COARSE GRID SEARCH (mode constraint enforced)
# Search over theta, derive k from (k-1)*theta = u_star
# ============================================================

best_score_p1 = np.inf
best_k_p1     = None
best_theta_p1 = None

for theta_try in np.linspace(0.70, 1.20, 20):
    k_try = (u_star / theta_try) + 1

    scores = []
    for _ in range(3):
        _, score = simulate_IS(k_try, theta_try)
        scores.append(score)

    avg_score = np.mean(scores)
    mode      = (k_try - 1) * theta_try
    marker    = " <- best" if avg_score < best_score_p1 else ""
    print(f"{theta_try:>8.3f} {k_try:>8.4f} {mode:>8.4f} {avg_score:>15.6e}{marker}")

    if avg_score < best_score_p1:
        best_score_p1 = avg_score
        best_k_p1     = k_try
        best_theta_p1 = theta_try



# ============================================================
# PHASE 2: FINE CROSS SEARCH (mode constraint dropped)
# Expand around Phase 1 best with fine step sizes
# k step = 0.005, theta step = 0.005
# Boundaries chosen to include the known optimum region
# ============================================================

best_score_p2 = np.inf
best_k_p2     = None
best_theta_p2 = None

theta_min = round(best_theta_p1 - 0.05, 3)
theta_max = round(best_theta_p1 + 0.05, 3)
k_min     = round(best_k_p1 - 0.15, 3)
k_max     = round(best_k_p1 + 0.15, 3)

theta_range = np.arange(theta_min, theta_max + 0.001, 0.005)
k_range     = np.arange(k_min,     k_max     + 0.001, 0.005)


for theta_try in theta_range:
    for k_try in k_range:
        scores = []
        for _ in range(3):
            _, score = simulate_IS(k_try, theta_try)
            scores.append(score)

        avg_score = np.mean(scores)
        mode      = (k_try - 1) * theta_try
```

```python
        marker    = " <- best" if avg_score < best_score_p2 else ""

        if marker or avg_score < best_score_p2 * 1.05:  # print only notable rows
            print(f"{theta_try:>8.3f} {k_try:>8.4f} {mode:>8.4f} "
                  f"{avg_score:>15.6e}{marker}")

        if avg_score < best_score_p2:
            best_score_p2 = avg_score
            best_k_p2     = k_try
            best_theta_p2 = theta_try


print("\n" + "="*80)
print(f"{'Optimization Stage':<25} | {'k':<8} | {'theta':<8} | {'Mode':<8} | {'Score':<15}")
print("-" * 80)

_, score = simulate_IS(k, theta)
# Stage 1: Geometric Baseline
print(f"{'Stage 1 (Geometric)':<25} | {k:<8.4f} | {theta:<8.4f} | {(theta-1)*k:<8.4f} | {score:>15.6e}")

# Phase 1: Coarse Grid Results
print(f"{'Phase 1 (Coarse Grid)':<25} | {best_k_p1:<8.4f} | {best_theta_p1:<8.4f} | {(best_k_p1-1)*best_theta_p1:<8.4f} | {best_score_p1:>15.6e}")

# Phase 2: Fine Cross Results (Best)
print(f"{'Phase 2 (Fine Cross)':<25} | {best_k_p2:<8.4f} | {best_theta_p2:<8.4f} | {(best_k_p2-1)*best_theta_p2:<8.4f} | {best_score_p2:>15.6e}")

print("="*80)
print(f"FINAL OPTIMUM: k = {best_k_p2:.4f}, theta = {best_theta_p2:.4f}")
print(f"FINAL ESTIMATED EAL: ${simulate_IS(best_k_p2, best_theta_p2)[0]:.6f}")
print(f"FINAL CONTEST SCORE: {best_score_p2:.6e}")
```

5) Results

Terminal Output

```
================================================================================
Optimization Stage | k | theta | Mode | Score
--------------------------------------------------------------------------------
Stage 1 (Analytical) | 4.9820 | 0.7990 | 3.1820 | 2.551241e-08
Phase 1 (Coarse Grid) | 4.3962 | 0.9368 | 3.1817 | 1.979728e-09
Phase 2 (Fine Cross) | 4.2910 | 0.9520 | 3.1330 | 1.086637e-09
================================================================================
FINAL OPTIMUM : k = 4.2910, theta = 0.9520
FINAL EAL : $0.001155
FINAL SCORE : 1.086637e-09
```

Discussion

The three-stage optimization successfully reduced the contest score from 2.55×10^{-8} in Stage 1 to 1.09×10^{-9} by the final Phase 2 search. This represents a total variance reduction of roughly $23\times$.

The calculated Expected Annual Loss (EAL) of **$0.001155 per turbine per year** is physically sound, despite its small scale with variance of 1.09×10^{-9} . This is explained by the Rayleigh model's prediction for the probability of a Category 4 hurricane ($v \geq 45$ m/s):

$$P(v \geq 45) = e^{-45^2/100} = e^{-20.25} \approx 1.6 \times 10^{-9}$$

While a single structural failure costs millions, the incredibly low annual probability of such an event leads to this small EAL. A standard Monte Carlo approach would likely sample $v < 45$ in every single run, returning a result of \$0 and falsely suggesting the risk is non-existent. By using Importance Sampling, we successfully captured this non-zero value with a standard error of just $\$1.04 \times 10^{-7}$, making the estimate precise to 4 significant figures using only $N = 100,000$ samples.